

Kubernetes, Docker Swarm & AWS Fargate

Complete Questions and Answers - Exercises 1 to 4

Compiled from this chat. Includes the background fact sheet, ERA baseline report, Chain of Verification audit, corrected report, assumption audit, and comparison matrix.

Exercise 1 - Background Knowledge Fact Sheet

Question: Generate a factual summary detailing the core architecture, scalability limits, setup complexity, and cost structures of Kubernetes, Docker Swarm, and AWS Fargate. Focus on raw specifications, documentation facts, and industry statistics. Do not write a comparison or recommendations yet.

Answer

Kubernetes

Kubernetes is an open-source container orchestration platform built around a control plane and worker-node architecture. The control plane includes the API server, scheduler, controller manager, and typically etcd. Worker nodes run containerized workloads as Pods, with kubelet and a container runtime. Kubernetes uses a declarative desired-state model.

Kubernetes provides abstractions for deployments, services, configuration, secrets, persistent storage, namespaces, autoscaling, rolling updates, health checking, and workload scheduling. Networking and storage commonly depend on CNI and CSI implementations.

Current Kubernetes v1.37 documentation describes large clusters satisfying these limits: up to 5,000 nodes, 110 Pods per node, 150,000 total Pods, and 300,000 total containers. Large clusters can also encounter infrastructure-provider quotas.

A self-managed Kubernetes environment requires control-plane and worker-node operation, networking, storage, security/RBAC, certificates, monitoring, upgrades, node lifecycle, and backup/recovery. Managed services move portions of these responsibilities to a cloud provider.

Kubernetes has no software license fee. Actual costs depend on compute, storage, networking/load balancers, IPs, data transfer, monitoring/logging, backup, operational engineering, and any managed-service charges.

Docker Swarm

Docker Swarm mode is Docker Engine's built-in clustering and orchestration capability. A swarm consists of manager and worker nodes. Managers maintain cluster state, schedule services, expose Swarm APIs, and use Raft consensus; workers execute assigned tasks.

A Swarm service defines desired workload state including image, replica count, resource limits, networking, ports, and update policies. Swarm includes service discovery, overlay networking, internal load balancing, TLS-secured node communication, rolling updates, and desired-state reconciliation.

Docker does not publish a simple current maximum worker-node number equivalent to Kubernetes' documented 5,000-node threshold. Docker recommends an odd number of managers for high availability and recommends a maximum of seven managers. Adding managers does not increase workload capacity and can reduce management-plane performance because of Raft acknowledgements.

Swarm functionality is integrated into Docker Engine and can be initialized with `docker swarm init`. Production environments still require host, networking, storage, backup, security, monitoring, manager quorum, and capacity management.

Docker Swarm mode itself does not impose a separate per-container orchestration charge. Infrastructure and administration are the principal runtime costs.

AWS Fargate

AWS Fargate is a serverless compute engine for containers within AWS. It supports Amazon ECS and Amazon EKS. AWS supplies and manages the underlying compute infrastructure while users specify workload resources.

Users remain responsible for container images, task or pod definitions, CPU/memory, IAM permissions, networking, storage requirements, logging, and application configuration.

Fargate scaling is controlled substantially through AWS account and regional service quotas rather than a customer-managed cluster-node ceiling. Capacity is governed by quotas, requested vCPU and memory, launch rates, networking resources, and related ECS/EKS limits.

Fargate removes the need to provision, patch, scale, or maintain container-host operating systems, although supporting AWS services still require configuration.

Fargate has no upfront compute infrastructure cost. Pricing primarily reflects requested vCPU, memory, operating system, CPU architecture, storage, and workload duration. Additional AWS services such as load balancing, data transfer, CloudWatch, ECR, storage, NAT Gateway, and databases can add charges. Fargate Spot offers interruptible capacity at a discount for suitable workloads.

Exercise 2 - ERA Baseline Research Report

Question: Use the ERA (Expectation, Role, Action) framework to produce an initial, unverified comparison report. Expectation: detailed comparison with H2 headers for Introduction, Tech Overviews, Detailed Comparison, and Final Verdict; no summary tables. Role: Lead Systems Architect. Action: write the report using the Exercise 1 background facts.

Answer - ERA Structured Research Prompt

EXPECTATION: I expect a detailed comparison report comparing Kubernetes, Docker Swarm, and AWS Fargate, structured with Introduction, Tech Overviews, Detailed Comparison, and Final Verdict. Do not write summary tables yet.

ROLE: Act as a Lead Systems Architect.

ACTION: Write the comparison report based on the Exercise 1 background facts.

Baseline Report

Introduction

Kubernetes, Docker Swarm, and AWS Fargate all support containerized application deployment, but they represent substantially different approaches to container infrastructure and orchestration. Kubernetes is a comprehensive orchestration platform designed to manage distributed container workloads across clusters. Docker Swarm provides container orchestration directly through Docker Engine with a smaller architectural footprint. AWS Fargate takes a managed-compute approach in which AWS operates the underlying container hosts while customers concentrate primarily on workload configuration. The major distinctions concern architecture, scalability, operational responsibility, setup complexity, ecosystem flexibility, and cost model.

Tech Overviews - Kubernetes

Kubernetes follows a control-plane and worker-node architecture. Its control plane manages scheduling, cluster state, orchestration, and reconciliation, while worker nodes execute application Pods. Its declarative architecture lets administrators define desired application state. Kubernetes supports service discovery, load balancing, rolling deployments, autoscaling, persistent storage, configuration management, namespaces, and RBAC. Its documented large-cluster thresholds make it suitable for substantial horizontal scaling, while self-management introduces networking, storage, security, monitoring, upgrades, backups, and node lifecycle responsibilities.

Tech Overviews - Docker Swarm

Docker Swarm provides native clustering and orchestration through Docker Engine. Managers maintain desired cluster state, schedule services, and participate in Raft consensus; workers execute assigned tasks. Swarm provides service discovery, overlay networking, load balancing, rolling updates, TLS-secured communication, and desired-state management. Its integration with Docker Engine reduces the number of distinct orchestration components, although underlying servers and production infrastructure still require administration.

Tech Overviews - AWS Fargate

AWS Fargate allows organizations to define container workloads and required resources while AWS operates the underlying compute infrastructure. It supports Amazon ECS and Amazon EKS. Customers remain responsible for images, CPU/memory, IAM, networking, logging, and supporting services. Scaling is influenced by AWS service quotas and allocated resources rather than customer-managed worker nodes. Pricing is consumption-oriented.

Detailed Comparison

Architecturally, Kubernetes provides the most extensive orchestration framework; Docker Swarm uses a smaller manager-worker architecture integrated into Docker Engine; Fargate abstracts the worker-host layer. Kubernetes

has explicit documented large-cluster limits, Docker Swarm expands workload capacity through workers while keeping the manager Raft group relatively small, and Fargate scales through managed AWS capacity and service quotas. Self-managed Kubernetes has substantial setup and operational responsibilities. Docker Swarm reduces initial orchestration setup but retains infrastructure ownership. Fargate removes customer-managed worker servers but still requires ECS/EKS, IAM, VPC, logging, storage, and related AWS configuration. Kubernetes and Swarm have no separate orchestration license charge, while Fargate uses consumption-based pricing. Kubernetes and Swarm can run across compatible infrastructure environments; Fargate is an AWS service.

Final Verdict

The baseline shows three distinct operating models. Kubernetes provides rich orchestration capabilities, substantial documented scalability, and infrastructure flexibility, with greater operational complexity when self-managed. Docker Swarm provides a compact Docker-integrated orchestration architecture while retaining infrastructure responsibility. AWS Fargate abstracts host infrastructure and uses a consumption-based model, reducing direct server administration while tying workloads to AWS services. No universal winner can be established from the background facts alone.

Exercise 3 - Chain of Verification (CoVe)

Question: Read the baseline draft and formulate exactly four specific verification questions that can be answered with absolute facts to audit claims, numbers, and limitations.

Answer - Four Verification Questions

1. Does the current official Kubernetes documentation still state support for clusters of up to 5,000 nodes, 150,000 total Pods, 300,000 total containers, and 110 Pods per node?
2. Does current Docker documentation recommend using an odd number of manager nodes and a maximum of seven managers in a Swarm cluster because of Raft consensus overhead?
3. Does AWS officially confirm that Fargate removes the need for customers to provision, manage, patch, and scale the underlying container host infrastructure, while customers still manage task or pod configuration, IAM, networking, and application settings?
4. Is Fargate currently billed primarily according to requested vCPU, memory, operating system/architecture where applicable, storage, and workload execution duration, with additional AWS services charged separately?

Question: Answer each of the four verification questions one-by-one, relying strictly on verified developer documentation facts.

Answer - Verification Results

- 1. Kubernetes scalability limits - Verified.** Kubernetes v1.37 documentation states clusters are designed to support no more than 5,000 nodes, 110 Pods per node, 150,000 total Pods, and 300,000 total containers.
- 2. Docker Swarm manager limitation - Verified with clarification.** Docker recommends an odd number of managers and recommends a maximum of seven manager nodes. Seven is a recommendation, not a hard technical maximum; Docker administration guidance states there is no fixed limit on manager nodes. Additional managers can reduce write performance because more Raft acknowledgements are required.
- 3. AWS Fargate operational responsibility - Verified.** AWS confirms that Fargate provisions and patches the underlying infrastructure. Customers still manage application and workload configuration, including networking and task/pod resource settings.
- 4. AWS Fargate pricing model - Verified.** Fargate pricing is based on requested vCPU, memory, operating system, CPU architecture, storage resources, and duration. Billing runs from image download start until task or pod termination, rounded to the nearest second, subject to minimum durations. Additional AWS services may incur separate charges.

Verification result: all four core baseline claims are supported. The principal correction is that Docker's seven-manager figure is a recommended maximum, not a hard platform limit.

Question: Review the original baseline draft and rewrite the report incorporating the verified corrections. Ensure the final report is fully accurate and resolved.

Answer - Corrected Final Report

Introduction

Kubernetes, Docker Swarm, and AWS Fargate provide different approaches to deploying and operating containerized applications. Kubernetes uses a control-plane and worker-node architecture; Docker Swarm provides orchestration through Docker Engine using managers and workers; AWS Fargate is a serverless container compute service for ECS and EKS that removes customer management of underlying compute hosts.

Kubernetes

Kubernetes is open source and uses declarative desired-state reconciliation. Current Kubernetes v1.37 documentation states support for clusters satisfying all of these limits: no more than 5,000 nodes, 110 Pods per node, 150,000 total Pods, and 300,000 total containers. Self-managed Kubernetes requires networking, storage, security, monitoring, control-plane availability, worker-node management, upgrades, and backup. Kubernetes itself has no software license fee; deployment costs come from infrastructure, operations, and any managed service used.

Docker Swarm

Docker Swarm mode is Docker Engine's native clustering and orchestration capability. Managers maintain cluster state and use Raft consensus; workers execute tasks. Docker recommends an odd number of managers and recommends a maximum of seven managers. Critically, seven is an architectural recommendation, not a hard technical limit. Worker nodes can be added separately to expand workload capacity. Swarm has no separate per-container orchestration charge, while hosts and supporting infrastructure remain the organization's responsibility.

AWS Fargate

AWS Fargate is a serverless compute engine supporting Amazon ECS and Amazon EKS. AWS provisions and patches the underlying infrastructure. Customers still manage workload configuration, networking, permissions, images, CPU/memory, logging, and related services. Scaling is governed by AWS service quotas and resource availability rather than customer-managed worker-node count. Pricing is consumption-based using requested vCPU, memory, operating system, CPU architecture, storage, and duration. Additional AWS services can add charges.

Detailed Comparison

Kubernetes provides extensive orchestration and explicit documented large-cluster limits. Docker Swarm uses a smaller Docker-integrated manager-worker model and separates manager consensus from worker workload capacity. Fargate abstracts host infrastructure and scales within AWS quotas and capacity. Self-managed Kubernetes generally carries substantial cluster administration. Swarm has fewer distinct orchestration components but still requires infrastructure management. Fargate removes host provisioning and patching while leaving application, network, IAM, and service configuration with the customer. Kubernetes and Swarm have no separate orchestration charge; Fargate is pay-per-use.

Final Verdict

The verified evidence describes three different operating models rather than a universal winner. Kubernetes emphasizes orchestration depth, scale, and infrastructure flexibility; Docker Swarm emphasizes Docker-integrated orchestration with retained host responsibility; Fargate emphasizes managed container compute and consumption-based pricing. Selection requires workload-specific criteria such as scale, expertise, infrastructure control, cloud strategy, availability requirements, and expected economics. The principal factual correction from the baseline is that Docker's seven-manager figure is a recommendation rather than an absolute technical platform limit.

Exercise 4 - Assumption Audit, Matrix & Executive Summary

Question: Analyze the finalized technology comparison. Identify three hidden assumptions the writer makes regarding the technologies and detail whether these assumptions introduce bias.

Answer - Assumption Audit

1. More infrastructure control implies greater operational complexity.

The report associates Kubernetes' extensive control with significant operational responsibility and Fargate's managed infrastructure with reduced host-management effort. This can introduce moderate bias because operational complexity also depends on deployment model and organizational capability. Managed Kubernetes can remove many control-plane tasks, while Fargate can still be complex through IAM, networking, observability, security, and application architecture.

2. Fewer orchestration components make Docker Swarm simpler to operate.

This introduces some bias if fewer components are interpreted as inherently easier production operation. Production complexity also depends on availability, monitoring, backup, networking, storage, troubleshooting, security, expertise, and workload requirements.

3. Technical capability and documented scalability are central factors in platform suitability.

The report gives substantial attention to Kubernetes scalability. This can create selection bias toward technical scale, even though many organizations will never approach those limits and may prioritize staffing, cloud commitments, compliance, portability, operational maturity, and total cost of ownership.

Audit conclusion: the report is largely factual, but decision criteria should treat scalability, simplicity, operational responsibility, infrastructure control, and cost independently rather than assuming more scale/control is always better or fewer components are automatically preferable.

Question: Create a markdown table comparing Kubernetes, Docker Swarm, and AWS Fargate across Technology, Scalability, Setup Complexity, Operational Overhead, and Cost Model.

Answer - Comparison Matrix

Technology	Scalability	Setup Complexity	Operational Overhead	Cost Model
Kubernetes	High	High	High	Free/open-source + infrastructure costs
Docker Swarm	Medium	Medium	Medium	Free orchestration + infrastructure costs
AWS Fargate	High	Low	Low	Pay-per-use

Note: High/Medium/Low ratings are qualitative classifications for the exercise, not official vendor ratings.

Executive Summary - 150 Words

Kubernetes, Docker Swarm, and AWS Fargate represent three distinct approaches to running containerized workloads. Kubernetes offers extensive orchestration capabilities, documented support for very large clusters, and broad infrastructure flexibility, but self-managed deployments require significant expertise and operational effort. Docker Swarm integrates orchestration directly into Docker Engine, providing a smaller architectural footprint and moderate operational demands while leaving organizations responsible for the underlying hosts and infrastructure.

AWS Fargate abstracts server provisioning and host maintenance through a managed, serverless container compute model for ECS and EKS. Its scaling depends on AWS service quotas and capacity, while pricing is based primarily on requested resources and execution duration. The analysis also identifies potential bias in assuming that greater control necessarily means greater complexity, fewer components always mean simpler operations, or higher scalability automatically makes a platform more suitable. Final selection should therefore reflect workload scale, team expertise, infrastructure control, cloud strategy, availability, and total cost.

Documentation references used during verification: Kubernetes large-cluster best practices; Docker Swarm node and administration guides; AWS ECS/Fargate shared responsibility and Fargate pricing documentation.